

category: avalonia

tags: [payment, cash, voucher, dto, immutable, event-args, navigation]

severity: high

created: 2026-07-22

updated: 2026-07-22

project-origin: iPGSv4 (PAYMENT_KIOSK_VERTICAL)

Số tiền đã trả một phần (cash/voucher) bị reset về 0 khi quay lại màn thanh toán

Tình huống gặp phải

Kiosk ô tô/xe máy: user thanh toán tiền mặt hoặc voucher nhưng chỉ đủ MỘT PHẦN số tiền ($\text{remain} > 0$). Sau đó quay lại màn chọn phương thức thanh toán (bấm Back, hoặc thoát ra rồi vào lại), số tiền "Đã trả" hiển thị **0** hoặc sai — mất luôn phần đã thanh toán dù server/DB đã ghi nhận đúng.

Triệu chứng / Lỗi

- Không có exception — chỉ là hiển thị sai số liệu (silent data bug).
- Xảy ra ở CẢ 2 nhánh: ô tô (CarInforDetail → PaymentInfo) và xe máy (MotoCash/MotoVoucher → MotorPayment).

Nguyên nhân gốc rễ (Root Cause)

Có 2 lớp nguyên nhân độc lập, cả hai đều thuộc pattern chung: "state tiền đã trả được giữ CỤC BỘ trong 1 object, nhưng object đó bị thay bằng bản MỚI/CŨ không mang theo state đó":

1. Re-fetch tạo DTO mới bỏ qua tiền đã trả (ô tô):

`CarInforDetailViewModel.LoadPaymentVehicleAsync()` và

`CarLocationViewModel.LoadPaymentVehicleAsync()` gọi

`GetEntryByIdAsync()` (API TRẢ VỀ ĐÚNG `detail.Amount` = tiền đã trả), nhưng

khi dựng `PaymentVehicleDto(entry, detail.Charge)` lại KHÔNG truyền

`detail.Amount` vào — constructor cũ luôn khởi tạo `_totalPaid = 0`. Mỗi lần

user quay lại rồi bấm "Thanh toán" lại → object hoàn toàn mới, mất sạch phần đã trả trước đó (dù server vẫn nhớ).

Đồng thời `EntryDetailDto.GetRemain() => IsPaid ? 0 : Charge` cũng bỏ qua `Amount` khi chưa `IsPaid` — cùng một lỗi tư duy, khác chỗ.

2. Event thông báo "quay lại"/"hoàn tất" không mang theo state cập nhật (xe máy):

`MotoCashViewModel/MotoVoucherViewModel` tích lũy tiền đã trả vào field cục bộ (`_paidSoFar / _payment.TotalPaid`), nhưng `BackRequested/PaymentCompleted` là `EventHandler?` KHÔNG tham số. Handler ở `MainWindow.axaml.cs` closure lại biến `vehicle/vehicleDto` GỐC (chụp tại thời điểm mở màn Cash/Voucher, TRƯỚC khi có bất kỳ khoản thanh toán một phần nào) để `NavigateTo(MotorBranch, ...)` — vô tình dùng bản chưa cập nhật.

Giải pháp

1. Thêm tham số `paidAmount` (optional, default 0) vào constructor `PaymentVehicleDto` (namespace `Models.Payment`, dùng cho ô tô) để set `_totalPaid` ngay từ đầu; truyền `detail.Amount` ở MỌI nơi dựng DTO từ API.
2. Sửa `EntryDetailDto.GetRemain()` → `Math.Max(0, Charge - Amount)` thay vì `IsPaid ? 0 : Charge`.
3. Đổi `EventHandler?` → `EventHandler<long>?` cho `BackRequested/PaymentCompleted` ở các ViewModel tích lũy tiền cục bộ (`MotoCash/MotoVoucher`), invoke kèm giá trị tổng đã trả hiện tại (`_paidSoFar/_payment.TotalPaid`).
Sửa lại handler ở `MainWindow.axaml.cs` để dựng DTO mới từ giá trị event args thay vì tái sử dụng biến closure gốc.
4. Với các ViewModel hiển thị `Remain/Paid` là computed property đọc thẳng từ DTO (không phải `ObservableProperty`) — PHẢI tự gọi `OnPropertyChanged(nameof(Remain))` sau khi gọi callback cộng dồn tiền, nếu không UI không tự refresh (thiếu ở `CarVoucherViewModel`, đã có sẵn ở `CarCashViewModel`).

Áp dụng lại (How to reuse)

- Khi thấy 1 DTO thanh toán được TRUYỀN QUA NHIỀU MÀN HÌNH bằng `NavigateTo(route, dto)` và có khả năng trả MỘT PHẦN (cash nhiều tờ, voucher không đủ) → luôn kiểm tra: (a) DTO có state mutable dùng chung 1 instance xuyên suốt hay bị "dựng lại" ở mỗi lần navigate? (b) nếu dựng lại, nguồn dữ liệu ban đầu (API/detail) có field "đã trả" không, và có được truyền vào constructor không?
- Khi thấy `EventHandler?` (không tham số) dùng để báo "user back"/"hoàn tất" từ 1 ViewModel có state tích lũy cục bộ → kiểm tra handler ở nơi subscribe có đang dùng lại biến closure GỐC (chụp trước khi có state mới) hay không.
- Property `Remain/Paid` dạng computed (get-only, đọc từ field khác) trong `ObservableObject` KHÔNG tự phát `PropertyChanged` — phải gọi thủ công sau mỗi lần state nguồn thay đổi.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Có 2 class `PaymentVehicleDto` trùng tên khác namespace trong cùng project (`Models.PaymentVehicleDto` — immutable, dùng nhánh xe máy; `Models.Payment.PaymentVehicleDto` — mutable qua `AddPayment()`, dùng nhánh ô tô). Alias rõ ràng (`CarPaymentVehicleDto`/`MotoPaymentVehicleDto`) khi cả hai xuất hiện cùng file, tránh nhầm lẫn fix sai bên.
- ⚠ Sửa `EntryDetailDto.GetRemain()` an toàn vì method này hiện KHÔNG có nơi gọi trực tiếp nào khác trong code (chỉ định nghĩa) — nhưng vẫn nên grep lại trước khi sửa method public dùng chung, vì call site mới có thể xuất hiện sau.

Tham chiếu

- Project liên quan: `iPGSv4 (IPGS.Kiosk.Avalonia)` — branch `PAYMENT_KIOSK_VERTICAL`
- File liên quan: `Models/Payment/PaymentVehicleDto.cs`,
`Services/Abstractions/IKioskApiService.cs`,
`ViewModels/Car/CarInforDetailViewModel.cs`,
`ViewModels/Car/CarLocationViewModel.cs`,
`ViewModels/Car/Payment/CarVoucherViewModel.cs`,
`ViewModels/Moto/MotoCashViewModel.cs`,
`ViewModels/Moto/MotoVoucherViewModel.cs`,
`Views/MainWindow.axaml.cs`